

ПОЛНОЕ ОПИСАНИЕ ИНТЕРФЕЙСА MOZART ERP (ФИНАЛЬНАЯ ВЕРСИЯ)

ОГЛАВЛЕНИЕ

1. Архитектура клиент-серверного взаимодействия
2. Этапы запуска клиента
3. Безопасность и протокол обмена
4. Хранение интерфейса в базе данных
5. Загрузка форм и модулей
6. Кэширование на клиенте
7. Роли и интерфейсные наборы
8. Fluent API для GUI
9. Взаимодействие форм
10. Примеры

1. АРХИТЕКТУРА КЛИЕНТ-СЕРВЕРНОГО ВЗАИМОДЕЙСТВИЯ

Layer 3 (Клиент):

- Крохотный модуль-загрузчик (~200KB)
- Приложенный JSON-манифест с описанием Layer 2 (URL, порт, fingerprint)
- Не знает о PostgreSQL, не содержит SQL-запросов
- Весь интерфейс получает с сервера
- Кэширует формы, модули, метаданные локально

Layer 2 (Сервер):

- Python-приложение (FastAPI / aiohttp)
- Знает PostgreSQL, выполняет все запросы
- Хранит в БД: интерфейс, формы, меню, права
- Отдаёт клиенту: модули, формы, данные
- Генерирует ключи шифрования

2. ЭТАПЫ ЗАПУСКА КЛИЕНТА

Этап 1. DISCOVERY (Получение манифеста)

Клиент → Сервер: GET /api/manifest

Сервер → Клиент: JSON с описанием версии, модулей, публичным ключом

Манифест содержит:

```
{  
  "server_version": "2.1.0",
```

```
"protocol_version": "1.0",
"modules": {
  "core": {"version": "2.1.0", "hash": "sha256:abc123..."},
  "docflow": {"version": "1.5.0", "hash": "sha256:def456..."},
  "accounting": {"version": "0.9.0", "hash": "sha256:ghi789..."}
},
"public_key": "-----BEGIN PUBLIC KEY-----...",
"features": ["docflow", "accounting", "crypto"]
}
```

Клиент сравнивает версии модулей с локальным кэшем.
Если версия изменилась — модуль будет перезагружен.

Этап 2. HANDSHAKE (Обмен ключами)

Клиент генерирует свою ключевую пару (асинхронную).

Клиент → Сервер: client_public_key + nonce

Сервер генерирует сессионный ключ (синхронный).

Сервер → Клиент: session_key, зашифрованный client_public_key

Клиент расшифровывает session_key.

Теперь у обеих сторон есть общий session_key для быстрого шифрования.
Переход в защищённый режим. Весь дальнейший обмен шифруется.

Этап 3. AUTH (Аутентификация)

Клиент показывает окно входа (логин/пароль).

Данные шифруются session_key.

Клиент → Сервер: POST /api/auth/login (зашифровано)

Сервер расшифровывает, проверяет через PostgreSQL (auth.users + crypt()).

Сервер проверяет роли пользователя (auth.userrolecontext).

Сервер → Клиент: токен сессии + интерфейсный набор пользователя.

Этап 4. SESSION (Загрузка интерфейса)

Клиент → Сервер: GET /api/ui/bootstrap (с токеном)

Сервер определяет интерфейсную роль пользователя.

Сервер → Клиент: минимальный интерфейсный набор:

- Меню (meta.menu — пункты, доступные этой роли)
- Панели инструментов
- Список доступных форм
- Права (auth.permissions для этой роли)
- Настройки интерфейса (тема, язык, раскладка)

Клиент строит главное окно на основе полученного набора.

Этап 5. WORK (Работа)

Пользователь работает в интерфейсе.

При открытии формы:

1. Проверяется кэш клиента
2. Если формы нет в кэше — запрос к серверу
3. Сервер отдаёт описание формы (контролы, привязки)
4. Клиент строит форму динамически
5. Данные подгружаются по мере необходимости

3. БЕЗОПАСНОСТЬ И ПРОТОКОЛ ОБМЕНА

Защита:

- Discovery: проверка fingerprint сервера (зашифрован в манифесте клиента)
- Handshake: асинхронное шифрование (ed25519 / x25519)
- Session: синхронное шифрование (AES-256-GCM) на session_key
- Auth: пароль передаётся ТОЛЬКО в зашифрованном виде
- Data: все запросы и ответы шифруются session_key

Хранение на клиенте:

- Токен сессии — в защищённом хранилище ОС
- Кэш модулей и форм — в зашифрованной SQLite базе
- Ключи сессии — в памяти, никогда на диск

4. ХРАНИЛИЩЕ ИНТЕРФЕЙСА В БАЗЕ ДАННЫХ

Весь интерфейс хранится в meta-таблицах. Никакого жёсткого кода UI.

meta.menu (существующая):

id, roleid, parentid, cname, calias, identitytypeid, isortorder

Определяет пункты меню для каждой роли.

roleid — ссылка на auth.roles.

identitytypeid — ссылка на meta.entitytypes (какой тип сущности открыть).

Пример:

Роль "Бухгалтер" → пункт "Счета" → identitytypeid = CHART_ACCOUNT

Роль "Кадровик" → пункт "Персоны" → identitytypeid = person

meta.forms (существующая):

id, identitytypeid, calias, cname, mstate_json

mstate_json — JSON-описание формы:

```
{
  "title": "Карточка персоны",
  "tabs": [
    {
      "name": "Основное",
      "controls": [
        {"type": "textbox", "name": "clastname", "label": "Фамилия", "bind":
"fields.clastname"},
        {"type": "textbox", "name": "cfirstname", "label": "Имя", "bind": "fields.cfirstname"},
        {"type": "datebox", "name": "dbirthdate", "label": "Дата рождения", "bind":
"fields.dbirthdate"}
      ]
    }
  ]
}
```

```

    ]
  },
  {
    "name": "Файлы",
    "controls": [
      {"type": "grid", "name": "files", "bind": "daughter_table.FILES",
        "columns": [
          {"field": "file_data_name", "header": "Файл", "width": 250},
          {"field": "mime_type", "header": "Тип", "width": 150}
        ]
      }
    ]
  }
],
"toolbar": [
  {"text": "Сохранить", "action": "save", "icon": "save"},
  {"text": "Заккрыть", "action": "close", "icon": "close"}
],
"size": {"width": 800, "height": 600}
}

```

meta.form_elements (существующая):

Детальное описание каждого элемента формы.

Альтернатива mstate_json для сложных форм.

Связь форм с ролями:

Через meta.menu: пункт меню → форма.

Или через отдельную таблицу meta.form_role_bindings (если нужно).

5. ЗАГРУЗКА ФОРМ И МОДУЛЕЙ

Порядок загрузки формы:

1. Пользователь нажимает пункт меню или кнопку
2. Клиент проверяет локальный кэш:
 - Если форма уже загружена и версия не изменилась → показать из кэша
 - Если нет → запрос к серверу
3. Клиент → Сервер: GET /api/ui/form/{form_calias}?entity_id={instance_id}
4. Сервер:
 - Читает meta.forms.mstate_json
 - Если форма привязана к типу сущности (ientitytypeid) — подставляет реальные имена полей из meta.fields
 - Отдаёт JSON с полным описанием формы + данные для привязки
5. Клиент динамически строит форму из JSON:
 - Создаёт контролы
 - Настраивает привязки (DataBinding)
 - Загружает данные (если передан instance_id)
 - Показывает форму

Загрузка модулей:

При старте (после discovery):

1. Клиент сравнивает версии модулей из манифеста с локальным кэшем
2. Для каждого изменённого модуля:
 - GET /api/modules/{module_name}/{version}
 - Сервер отдаёт .zip с Python-байткодом (.pyc) или исходниками
 - Клиент сохраняет в зашифрованный кэш
3. Модули загружаются динамически (importlib)

Обновление модулей:

- Сервер может отправить уведомление (WebSocket) о новой версии
- Клиент перезагружает модуль без перезапуска приложения
- Формы, построенные на старом модуле, продолжают работать
- Новые экземпляры форм используют новую версию

6. КЭШИРОВАНИЕ НА КЛИЕНТЕ

Кэш клиента (SQLite, зашифрована session_key):

Таблица module_cache:

module_name, version, data (blob), loaded_at

Таблица form_cache:

form_alias, form_json, entitytype_id, cached_at

Таблица meta_cache:

entitytypes, fields, composition_roles, ...

Обновляется при изменении на сервере (уведомление или таймаут)

Таблица data_cache:

entity_alias, instance_id, version_id, data_json, cached_at

Кэш недавно использованных данных.

Стратегия кэширования:

- Формы: кэшируются до изменения метаданных (версия формы)
- Метаданные: кэшируются с TTL = 5 минут или по уведомлению
- Данные: кэшируются до изменения (оптимистичный lock) или 30 секунд
- Модули: кэшируются до изменения версии в манифесте

Инвалидация кэша:

- Сервер → Клиент: WebSocket-уведомление "metadata_changed"
- При ошибке 409 Conflict: "data_changed" → перезапросить
- По таймауту (для данных)

7. РОЛИ И ИНТЕРФЕЙСНЫЕ НАБОРЫ

=====

Пользователь входит в систему. У него есть запись в userrolecontext:

userid → роль → балансовая единица

Роль имеет тип (role_type):

CONFIG — конфигуратор (видит всё)

USER — пользователь

INTERFACE — интерфейсная роль

Интерфейсный набор определяется интерфейсной ролью:

- Меню (meta.menu WHERE roleid = интерфейсная_роль)
- Доступные формы (через меню или form_role_bindings)
- Права (auth.permissions WHERE role_id = роль)
- Видимые типы сущностей (через permissions)
- Настройки отображения (тема, язык)

Пользователь может иметь несколько ролей в разных БЕ.

При входе выбирает: "Войти как Бухгалтер (БЕ001)".

Интерфейс перестраивается под выбранную роль:

- Меню фильтруется
- Список доступных действий ограничивается permissions
- Поля ввода — read-only если нет права UPDATE

=====

8. FLUENT API ДЛЯ GUI (КРАТКО)

=====

(Подробно описано в документе "GUI_Fluent_API_v1.1", здесь кратко)

ERPMozart.gui:

main_window, forms, controls, bindings, events, theme, translator

Формы немодальные. Знают родителя. Изменения — в локальной модели.

Сохранение — по кнопке "Сохранить".

Контролы: textbox, numberbox, datebox, checkbox, combobox, memo,
file_picker, reference, grid, tree, button, label

DataBinding: bind(control, entity_path, field_name)

save() — сохранить BCE dirty-контролы в БД

rollback() — откатить

is_modified() — есть ли изменения

=====

9. ВЗАИМОДЕЙСТВИЕ ФОРМ

=====

=====

Формы немодальные. Каждая знает родителя.

Открытие дочерней формы:

```
gui.forms.open('person_edit', parent=self, person_id=123)
```

Возврат результата:

```
child_form.return_result({'action': 'saved', 'person_id': 123})
```

Родитель получает:

```
parent_form.on_child_result(child_form, result)
```

События (между формами без прямой связи родитель-потомок):

```
gui.events.subscribe(self, 'item_selected', 'person_list', callback)
gui.events.emit('item_selected', {'person_id': 123})
```

=====

=====

10. ПРИМЕРЫ

=====

=====

Пример 1: Пользователь входит как Бухгалтер (БЕ001)

1. Клиент прошёл discovery, handshake, auth
2. Сервер определил роль "Бухгалтер" в БЕ001
3. Отдал меню: "План счетов", "Проводки", "Отчёты"
4. Отдал права: CREATE, READ, UPDATE на CHART_ACCOUNT
5. Клиент построил главное окно с этим меню

Пример 2: Бухгалтер открывает карточку счёта

1. Нажимает "План счетов" → "Счёт 10.01"
2. Клиент проверяет кэш формы "chart_account_edit"
3. Если нет в кэше — GET /api/ui/form/chart_account_edit?entity_id=45
4. Сервер отдаёт JSON формы + данные счёта 10.01
5. Клиент строит форму: текстовые поля, таблицу проводок, тулбар
6. Данные привязываются через DataBinding
7. Форма показана

Пример 3: Бухгалтер меняет наименование счёта

1. Вводит новое имя в textbox
2. Контрол становится dirty (is_dirty=true)
3. Форма помечается is_modified=true
4. Нажимает "Сохранить"
5. gui.bindings.save():
 - Для версионной сущности: закрывает старую версию, создаёт новую
 - Для неверсионной: UPDATE
6. Форма помечается is_modified=false

Пример 4: Бухгалтер открывает выписку по счёту

1. В карточке счёта нажимает "Выписка"

2. `gui.forms.open('account_statement', parent=current_form, account_id=45)`
3. Открывается новая форма поверх (немодальная)
4. Бухгалтер смотрит, закрывает
5. Родительская форма остаётся открытой

=====

=====

КОНЕЦ ОПИСАНИЯ

=====

=====